Use case reference

CROSSFADE, INFO 579 Project 2. 15 use cases from the Project 1 requirements analysis (RA). For each: the tables and columns it needs, which Project 2 file answers it, and, where single_table.sql answers it, the query's input and expected output and every test case. Expected outputs are computed in Python from the Project 1 sample data (oracle.py) and were matched by MySQL in the test run of 2026-11-06 13:45:36 (66 passed, 0 failed). Day counts use the fixed report date 2026-09-30.

Summary

UC	Use case	Tables it needs	Answered by
UC-01	Pay each artist a per-stream royalty	artist, track, play	single_table.sql: S1
UC-02	Route payments by country; pay only verified artists	artist	single_table.sql: S1
UC-03	Surface new uploads as readily as hits	track	single_table.sql: S2
UC-04	Track the upload pipeline; reject duplicate uploads	track, track_score	single_table.sql: S7
UC-05	Show the genre legend while listeners compose a blend	genre	single_table.sql: S3
UC-06	Bill subscribers by plan tier from their signup date	listener	single_table.sql: S4
UC-07	Share stations under the creator's display name	listener, station, play	multiple_table.sql / routines.sql (not written yet)
UC-08	Weight each model version by its measured accuracy	scoring_model, station	single_table.sql: S5, S6
UC-09	Find scoring results that predate a model redeployment	track_score, scoring_model	multiple_table.sql / routines.sql (not written yet)
UC-10	Define stations by ambiguity window; measure how long they stay in use	station, station_blend	single_table.sql: S6
UC-11	Place each recording between genres	track_score, genre_probability, genre	multiple_table.sql / routines.sql (not written yet)
UC-12	Fill a station with recordings that match its blend and window	station, station_blend, track_score, genre_probability, track	multiple_table.sql / routines.sql (not written yet)
UC-13	Learn listener taste from completions, saves and skips	play, track	multiple_table.sql / routines.sql (not written yet)
UC-14	Keep an exact royalty ledger	play, track, artist	multiple_table.sql / routines.sql (not written yet)
UC-15	Reject a duplicate upload	track	single_table.sql: E1

UC-01 Pay each artist a per-stream royalty

Question: What is each artist paid per qualifying stream?

Amounts actually owed need play, track and artist together (Task 3 or 4).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`artist`	`artist_id, stage_name, payout_rate`	8 rows	alone: partial
`track`	`track_id, artist_id`	15 rows	with a join
`play`	`track_id, seconds_played, royalty_amount`	31 rows	with a join

S1 | crossfade.artist | single_table.sql

BUSINESS NEED

CROSSFADE pays independent artists a per-stream royalty, and pays only verified artists through a channel chosen by their country (RA: Overview; Artist: Payout Rate, Country, Verified).

QUESTION

Who can be paid in this royalty run, who is on hold and why, and what does each artist earn per 1,000 qualifying streams?

INPUT

The Project 1 sample data in `crossfade.artist` (8 rows), read through these columns: `artist_id, country, joined_date, payout_rate, stage_name, verified`. Brief requirements met: (a) (c) (e).

```
SELECT artist_id,
       CONCAT(stage_name, ' (', IFNULL(country, '--'), ')') AS payee,
       ROUND(payout_rate * 1000, 2) AS usd_per_1000_streams,
       DATEDIFF('2026-09-30', joined_date) AS days_under_licence,
       CASE
           WHEN verified = 0 THEN 'HOLD: identity not verified'
           WHEN country IS NULL THEN 'HOLD: no payment country'
           ELSE 'PAYABLE'
       END AS payment_status
FROM crossfade.artist
ORDER BY CASE WHEN verified = 1 AND country IS NOT NULL THEN 0 ELSE 1 END,
         payout_rate DESC,
         artist_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

artist_id	payee	usd_per_1000_streams	days_under_licence	payment_status
1042	Velvet Static (US)	4.00	200	PAYABLE
1044	Low Orbit Choir (CA)	3.80	239	PAYABLE
1046	Ochre Parade (MX)	3.60	172	PAYABLE
1043	Mara Quell (GB)	3.50	253	PAYABLE
1047	Juno Fleet (DE)	3.00	151	PAYABLE

artist_id	payee	usd_per_1000_streams	days_under_licence	payment_status
1049	Kestrel Ave (JM)	3.40	33	HOLD: identity not verified
1045	Tin Harbour (AU)	3.20	215	HOLD: identity not verified
1048	Saltmarsh Tapes (--)	3.00	134	HOLD: identity not verified

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A verified artist with no payment country is held, not paid.

Why: the sample data never reaches this CASE branch (its one artist with no country is also unverified, and the first WHEN wins).

Input:

```
INSERT into artist: artist_id=9001, stage_name=Zero Country, country=NULL, joined_date=2026-09-01, payout_rate=0.0020, verified=1
```

Expected: artist 9001 reads 'HOLD: no payment country', payee 'Zero Country (--)'.

artist_id	payee	usd_per_1000_streams	days_under_licence	payment_status
1042	Velvet Static (US)	4.00	200	PAYABLE
1044	Low Orbit Choir (CA)	3.80	239	PAYABLE
1046	Ochre Parade (MX)	3.60	172	PAYABLE
1043	Mara Quell (GB)	3.50	253	PAYABLE
1047	Juno Fleet (DE)	3.00	151	PAYABLE
1049	Kestrel Ave (JM)	3.40	33	HOLD: identity not verified
1045	Tin Harbour (AU)	3.20	215	HOLD: identity not verified
1048	Saltmarsh Tapes (--)	3.00	134	HOLD: identity not verified
9001	Zero Country (--)	2.00	29	HOLD: no payment country

PASS Two payable artists on the same rate are ordered by artist_id.

Why: without a tie-break the order of equal rates is up to MySQL and could change between runs.

Input:

```
INSERT into artist: artist_id=900, stage_name=Aardvark Tie, country=US, joined_date=2026-01-01, payout_rate=0.0040, verified=1
```

Expected: artist 900 (rate 0.0040, same as 1042) is listed first, 1042 second.

artist_id	payee	usd_per_1000_streams	days_under_licence	payment_status
900	Aardvark Tie (US)	4.00	272	PAYABLE
1042	Velvet Static (US)	4.00	200	PAYABLE
1044	Low Orbit Choir (CA)	3.80	239	PAYABLE
1046	Ochre Parade (MX)	3.60	172	PAYABLE
1043	Mara Quell (GB)	3.50	253	PAYABLE
1047	Juno Fleet (DE)	3.00	151	PAYABLE
1049	Kestrel Ave (JM)	3.40	33	HOLD: identity not verified
1045	Tin Harbour (AU)	3.20	215	HOLD: identity not verified
1048	Saltmarsh Tapes (--)	3.00	134	HOLD: identity not verified

UC-02 Route payments by country; pay only verified artists

Question: Who is payable in this royalty run, who is on hold, and why?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`artist`	`artist_id, stage_name, country, joined_date, verified`	8 rows	alone: full

S1 | crossfade.artist | single_table.sql

BUSINESS NEED

CROSSFADE pays independent artists a per-stream royalty, and pays only verified artists through a channel chosen by their country (RA: Overview; Artist: Payout Rate, Country, Verified).

QUESTION

Who can be paid in this royalty run, who is on hold and why, and what does each artist earn per 1,000 qualifying streams?

INPUT

The Project 1 sample data in `crossfade.artist` (8 rows), read through these columns: `artist_id, country, joined_date, payout_rate, stage_name, verified`. Brief requirements met: (a) (c) (e).

```
SELECT artist_id,
       CONCAT(stage_name, ' (', IFNULL(country, '--'), ')') AS payee,
       ROUND(payout_rate * 1000, 2) AS usd_per_1000_streams,
       DATEDIFF('2026-09-30', joined_date) AS days_under_licence,
       CASE
           WHEN verified = 0 THEN 'HOLD: identity not verified'
           WHEN country IS NULL THEN 'HOLD: no payment country'
           ELSE 'PAYABLE'
       END AS payment_status
FROM crossfade.artist
ORDER BY CASE WHEN verified = 1 AND country IS NOT NULL THEN 0 ELSE 1 END,
         payout_rate DESC,
         artist_id;
```

EXPECTED OUTPUT: 8 row(s), in this order

artist_id	payee	usd_per_1000_streams	days_under_licence	payment_status
1042	Velvet Static (US)	4.00	200	PAYABLE
1044	Low Orbit Choir (CA)	3.80	239	PAYABLE
1046	Ochre Parade (MX)	3.60	172	PAYABLE
1043	Mara Quell (GB)	3.50	253	PAYABLE
1047	Juno Fleet (DE)	3.00	151	PAYABLE
1049	Kestrel Ave (JM)	3.40	33	HOLD: identity not verified
1045	Tin Harbour (AU)	3.20	215	HOLD: identity not verified
1048	Saltmarsh Tapes (--)	3.00	134	HOLD: identity not verified

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A verified artist with no payment country is held, not paid.

Why: the sample data never reaches this CASE branch (its one artist with no country is also unverified, and the first WHEN wins).

Input:

```
INSERT into artist: artist_id=9001, stage_name=Zero Country, country=NULL, joined_date=2026-09-01, payout_rate=0.0020, verified=1
```

Expected: artist 9001 reads 'HOLD: no payment country', payee 'Zero Country (--)'.

artist_id	payee	usd_per_1000_streams	days_under_licence	payment_status
1042	Velvet Static (US)	4.00	200	PAYABLE
1044	Low Orbit Choir (CA)	3.80	239	PAYABLE
1046	Ochre Parade (MX)	3.60	172	PAYABLE
1043	Mara Quell (GB)	3.50	253	PAYABLE
1047	Juno Fleet (DE)	3.00	151	PAYABLE
1049	Kestrel Ave (JM)	3.40	33	HOLD: identity not verified
1045	Tin Harbour (AU)	3.20	215	HOLD: identity not verified
1048	Saltmarsh Tapes (--)	3.00	134	HOLD: identity not verified
9001	Zero Country (--)	2.00	29	HOLD: no payment country

PASS Two payable artists on the same rate are ordered by artist_id.

Why: without a tie-break the order of equal rates is up to MySQL and could change between runs.

Input:

```
INSERT into artist: artist_id=900, stage_name=Aardvark Tie, country=US, joined_date=2026-01-01, payout_rate=0.0040, verified=1
```

Expected: artist 900 (rate 0.0040, same as 1042) is listed first, 1042 second.

artist_id	payee	usd_per_1000_streams	days_under_licence	payment_status
900	Aardvark Tie (US)	4.00	272	PAYABLE
1042	Velvet Static (US)	4.00	200	PAYABLE
1044	Low Orbit Choir (CA)	3.80	239	PAYABLE
1046	Ochre Parade (MX)	3.60	172	PAYABLE
1043	Mara Quell (GB)	3.50	253	PAYABLE
1047	Juno Fleet (DE)	3.00	151	PAYABLE
1049	Kestrel Ave (JM)	3.40	33	HOLD: identity not verified
1045	Tin Harbour (AU)	3.20	215	HOLD: identity not verified
1048	Saltmarsh Tapes (--)	3.00	134	HOLD: identity not verified

UC-03 Surface new uploads as readily as hits

Question: Which are the newest recordings listeners can stream?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track`	`track_id, title, duration, release_date, ingest_status`	15 rows	alone: full

S2 | crossfade.track | single_table.sql

BUSINESS NEED

a recording uploaded an hour ago must be as recommendable as one with a million plays (RA: Overview), and only scored recordings may be streamed (RA: Track).

QUESTION

What are the five newest recordings listeners can stream?

INPUT

The Project 1 sample data in `crossfade.track` (15 rows), read through these columns: `duration, ingest_status, release_date, title, track_id`. Brief requirements met: (a) (c) (d).

```
SELECT track_id,
       title,
       CONCAT(duration DIV 60, ':', LPAD(MOD(duration, 60), 2, '0')) AS run_time,
       release_date,
       DATEDIFF('2026-09-30', release_date) AS days_since_release
FROM crossfade.track
WHERE ingest_status = 'scored'
ORDER BY release_date DESC,
         track_id
LIMIT 5;
```

EXPECTED OUTPUT: 5 row(s), in this order

track_id	title	run_time	release_date	days_since_release
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92
4829	Tapeworm Blues	3:09	2026-06-05	117
4827	Stahlwerk	4:14	2026-05-25	128
4822	Hymn for a Server Room	4:28	2026-05-15	138

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS The newest upload stays off the shelf until it is scored.

Why: only scored recordings can be streamed, so date alone must not put a track on the shelf.

Input:

```
INSERT into track: track_id=9100, artist_id=1042, title=Fresh But Unscored, duration=200, release_date=2026-09-29,
audio_checksum=0x0101010101010101..., ingest_status=awaiting scoring
```

Expected: track 9100 (released 2026-09-29, awaiting scoring) is not in the result.

track_id	title	run_time	release_date	days_since_release
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92
4829	Tapeworm Blues	3:09	2026-06-05	117
4827	Stahlwerk	4:14	2026-05-25	128
4822	Hymn for a Server Room	4:28	2026-05-15	138

PASS Two releases on the same day are ordered by track_id, and run time is shown as m:ss.

Why: same-day releases tie on the date; 61 seconds checks the seconds are zero-padded.

Input:

```
INSERT into track: track_id=4800, artist_id=1043, title=Same Day, duration=61, release_date=2026-07-10, audio_checksum=0x0202020202020202...,
ingest_status=scored
```

Expected: 4800 then 4828 at the top; 4800's run_time is '1:01'.

track_id	title	run_time	release_date	days_since_release
4800	Same Day	1:01	2026-07-10	82
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92
4829	Tapeworm Blues	3:09	2026-06-05	117
4827	Stahlwerk	4:14	2026-05-25	128

PASS The shelf never shows more than five recordings.

Why: the sample data has exactly enough scored tracks that LIMIT 5 is never pushed.

Input:

```
INSERT into track: track_id=9200, artist_id=1044, title=New 0, duration=120, release_date=2026-09-20, audio_checksum=0x1010101010101010..., ingest_status=scored
```

```
INSERT into track: track_id=9201, artist_id=1044, title=New 1, duration=120, release_date=2026-09-21, audio_checksum=0x1111111111111111..., ingest_status=scored
```

```
INSERT into track: track_id=9202, artist_id=1044, title=New 2, duration=120, release_date=2026-09-22, audio_checksum=0x1212121212121212..., ingest_status=scored
```

Expected: 5 rows, newest first: track 9202 (released 2026-09-22).

track_id	title	run_time	release_date	days_since_release
9202	New 2	2:00	2026-09-22	8
9201	New 1	2:00	2026-09-21	9
9200	New 0	2:00	2026-09-20	10
4828	Afterimage Club	3:41	2026-07-10	82
4826	Plaza at Noon	3:18	2026-06-30	92

UC-04 Track the upload pipeline; reject duplicate uploads

Question: How many recordings sit at each stage of the upload pipeline?

Checking rule I5 needs track_score (Task 3).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track`	`track_id, release_date, audio_checksum, ingest_status`	15 rows	alone: partial
`track_score`	`track_id`	27 rows	with a join

S7 | crossfade.track | single_table.sql

BUSINESS NEED

every upload is scored before it can be streamed (RA: Track: Ingest Status), so a backlog in the upload pipeline must be visible.

QUESTION

How many recordings sit at each stage of the upload pipeline, and is scoring keeping up with their release dates?

INPUT

The Project 1 sample data in `crossfade.track` (15 rows), read through these columns: `ingest_status, release_date`. Brief requirements met: (a) (b) (c) (e).

```
SELECT ingest_status,
       COUNT(*) AS recordings,
       MIN(release_date) AS earliest_release,
       DATEDIFF(MIN(release_date), '2026-09-30') AS days_to_earliest_release,
       CASE
           WHEN ingest_status = 'scored' THEN 'streamable'
           WHEN ingest_status = 'rejected' THEN 'never streamable: rejected'
           WHEN MIN(release_date) <= '2026-09-30' THEN 'LATE: release date reached, still unscored'
           ELSE 'on time: release date still ahead'
       END AS release_status
FROM crossfade.track
GROUP BY ingest_status
ORDER BY ingest_status;
```

EXPECTED OUTPUT: 3 row(s), in this order

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-09-21	-9	LATE: release date reached, still unscored
scored	13	2026-02-25	-217	streamable
rejected	1	2026-09-22	-8	never streamable: rejected

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A new rejection is counted, and statuses stay in pipeline order, not alphabetical.

Why: ingest_status is an ENUM; ORDER BY must follow awaiting scoring, scored, rejected.

Input:

```
INSERT into track: track_id=9300, artist_id=1049, title=Clipped Master, duration=30, release_date=2026-09-28,
audio_checksum=0x0303030303030303..., ingest_status=rejected
```

Expected: rejected shows 2 recordings, earliest release 2026-09-22 (8 days ago), 'never streamable: rejected'.

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-09-21	-9	LATE: release date reached, still unscored
scored	13	2026-02-25	-217	streamable
rejected	2	2026-09-22	-8	never streamable: rejected

PASS An unscored recording whose release date is still ahead is on time, not late.

Why: the sample's one unscored recording is already past its release date (LATE), so the on-time branch is never reached.

Input:

```
UPDATE track SET release_date = '2026-10-15' WHERE ingest_status = 'awaiting scoring' (1 row(s) changed)
```

Expected: with its release moved to 2026-10-15, awaiting scoring reads 15 days ahead, 'on time: release date still ahead'.

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-10-15	15	on time: release date still ahead
scored	13	2026-02-25	-217	streamable
rejected	1	2026-09-22	-8	never streamable: rejected

PASS A release due on the report date itself, still unscored, is already late.

Why: the boundary: on its release day the recording must already be streamable, so 'reached' includes the day.

Input:

```
UPDATE track SET release_date = '2026-09-30' WHERE ingest_status = 'awaiting scoring' (1 row(s) changed)
```

Expected: with its release moved to 2026-09-30, awaiting scoring reads 0 days, 'LATE: release date reached, still unscored'.

ingest_status	recordings	earliest_release	days_to_earliest_release	release_status
awaiting scoring	1	2026-09-30	0	LATE: release date reached, still unscored
scored	13	2026-02-25	-217	streamable
rejected	1	2026-09-22	-8	never streamable: rejected

UC-05 Show the genre legend while listeners compose a blend

Question: Is every genre ready to show in the blend composer, in model class order?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`genre`	`genre_id, name, description`	10 rows	alone: full

S3 | crossfade.genre | single_table.sql

BUSINESS NEED

listeners compose station blends from the genre legend, which shows them each genre's description (RA: Genre).

QUESTION

Is every genre ready to show in the blend composer, listed in the order the scoring models output them?

INPUT

The Project 1 sample data in `crossfade.genre` (10 rows), read through these columns: `description, genre_id, name`. Brief requirements met: (a) (c) (e).

```
SELECT genre_id AS model_class_index,
       name AS label,
       CONCAT(name, ': ', IFNULL(description, '(no description)')) AS legend_line,
       IF(description IS NULL, 'HIDE: no description for the composer', 'READY') AS composer_status
FROM crossfade.genre
ORDER BY genre_id;
```

EXPECTED OUTPUT: 10 row(s), in this order

model_class_index	label	legend_line	composer_status
0	blues	blues: Twelve-bar forms, blue notes and call-and-response phrasing	READY
1	classical	classical: Orchestral and chamber writing in the Western art-music tradition	READY
2	country	country: Narrative songwriting with steel guitar, fiddle and twang	READY
3	disco	disco: Four-on-the-floor dance grooves with lush strings and bass	READY
4	hiphop	hiphop: Rhythmic vocal delivery over sampled or programmed beats	READY
5	jazz	jazz: Swing, improvisation and extended harmony	READY
6	metal	metal: Distorted guitars, heavy drums and aggressive dynamics	READY
7	pop	pop: Hook-driven songs built for broad appeal	READY
8	reggae	reggae: Offbeat skank rhythm and prominent bass lines	READY

model_class_index	label	legend_line	composer_status
9	rock	rock: Guitar-led songs with a driving backbeat	READY

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A genre with no description is flagged to be hidden from the blend composer.

Why: every genre in the sample data has a description, so the HIDE branch is never reached.

Input:

```
UPDATE genre SET description = NULL WHERE genre_id = 9 (1 row(s) changed)
```

Expected: genre 9 (rock) shows 'rock: (no description)' and 'HIDE: no description for the composer'.

model_class_index	label	legend_line	composer_status
0	blues	blues: Twelve-bar forms, blue notes and call-and-response phrasing	READY
1	classical	classical: Orchestral and chamber writing in the Western art-music tradition	READY
2	country	country: Narrative songwriting with steel guitar, fiddle and twang	READY
3	disco	disco: Four-on-the-floor dance grooves with lush strings and bass	READY
4	hiphop	hiphop: Rhythmic vocal delivery over sampled or programmed beats	READY
5	jazz	jazz: Swing, improvisation and extended harmony	READY
6	metal	metal: Distorted guitars, heavy drums and aggressive dynamics	READY
7	pop	pop: Hook-driven songs built for broad appeal	READY
8	reggae	reggae: Offbeat skank rhythm and prominent bass lines	READY
9	rock	rock: (no description)	HIDE: no description for the composer

UC-06 Bill subscribers by plan tier from their signup date

Question: What is the subscriber mix by plan tier, and how long has each tier been billing?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`listener`	`listener_id, email, plan_tier, signup_date`	8 rows	alone: full

S4 | crossfade.listener | single_table.sql

BUSINESS NEED

subscribers are billed by plan tier, each from their own signup date (RA: Listener: Plan Tier, Signup Date).

QUESTION

How many subscribers does each plan tier have, when did the first and latest join, and how long has the average subscriber been billing?

INPUT

The Project 1 sample data in `crossfade.listener` (8 rows), read through these columns: `plan_tier, signup_date`. Brief requirements met: (a) (b) (c).

```
SELECT plan_tier,
       COUNT(*) AS listeners,
       MIN(signup_date) AS first_signup,
       MAX(signup_date) AS latest_signup,
       ROUND(AVG(DATEDIFF('2026-09-30', signup_date))) AS avg_days_subscribed
FROM crossfade.listener
GROUP BY plan_tier
ORDER BY plan_tier DESC;
```

EXPECTED OUTPUT: 3 row(s), in this order

plan_tier	listeners	first_signup	latest_signup	avg_days_subscribed
premium	2	2026-04-02	2026-05-27	154
standard	3	2026-05-09	2026-07-08	116
free	3	2026-04-18	2026-08-15	104

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS A plan tier nobody is on does not appear in the subscriber mix.

Why: every tier has listeners in the sample data; GROUP BY must not invent an empty tier.

Input:

```
UPDATE listener SET plan_tier = 'standard' WHERE plan_tier = 'premium' (2 row(s) changed)
```

Expected: only standard and free, in tier order, standard with 5 listeners.

plan_tier	listeners	first_signup	latest_signup	avg_days_subscribed
standard	5	2026-04-02	2026-07-08	131
free	3	2026-04-18	2026-08-15	104

UC-07 Share stations under the creator's display name

Alone, listener only yields a list of names, which is not a real question. Needs station and play (Task 3).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`listener`	`listener_id, display_name`	8 rows	with a join
`station`	`station_id, listener_id, name`	8 rows	with a join
`play`	`listener_id, station_id`	31 rows	with a join

Input / expected output: not defined yet. This use case is answered in multiple_table.sql or routines.sql; its query, input and expected output are added when that file and its tests are written.

UC-08 Weight each model version by its measured accuracy

Question: Which model versions should carry the most weight, and which should a new station default to?

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`scoring_model`	`model_id, version_label, fusion_type, embedding_dimension, segment_length, training_corpus, holdout_accuracy, deployed_on`	5 rows	alone: full
`station`	`station_id, model_id`	8 rows	alone: partial

S5 | crossfade.scoring_model | single_table.sql

BUSINESS NEED

each scoring model version is weighted by its measured holdout accuracy, and versions trained on different corpora are never silently compared (RA: Scoring Model).

QUESTION

Which model version should a new station default to?

INPUT

The Project 1 sample data in `crossfade.scoring_model` (5 rows), read through these columns: `deployed_on, embedding_dimension, fusion_type, holdout_accuracy, segment_length, training_corpus, version_label`. Brief requirements met: (a) (c) (d).

```
SELECT version_label,
       CONCAT(fusion_type, ' / ', embedding_dimension, 'd / ', segment_length, 's / ', training_corpus) AS architecture,
       ROUND(holdout_accuracy * 100, 2) AS accuracy_pct,
       deployed_on
FROM crossfade.scoring_model
ORDER BY holdout_accuracy DESC,
         deployed_on DESC
LIMIT 1;
```

EXPECTED OUTPUT: 1 row(s), in this order

version_label	architecture	accuracy_pct	deployed_on
beardown_3sec	concat / 384d / 3s / GTZAN	78.87	2026-06-23

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS On equal holdout accuracy the more recently deployed version becomes the default.

Why: the sample's top version has no tie, so the deployed_on tie-break is never used for the default.

Input:

```
INSERT into scoring_model: model_id=6, version_label=tie_newer, fusion_type=gated, embedding_dimension=256, segment_length=10,
training_corpus=GTZAN, holdout_accuracy=0.7887, deployed_on=2026-09-01
```

Expected: tie_newer (accuracy 0.7887 like beardown_3sec, but deployed later, 2026-09-01) is the one default.

version_label	architecture	accuracy_pct	deployed_on
tie_newer	gated / 256d / 10s / GTZAN	78.87	2026-09-01

S6 | crossfade.station | single_table.sql

BUSINESS NEED

every station is evaluated against one nominated model version inside an ambiguity window (RA: Station; Scoring Model 1:N Station), so retiring or redeploying a version changes the stations that rely on it.

QUESTION

How many stations does each model version serve, how wide are their ambiguity windows, and since when has each version been relied on?

INPUT

The Project 1 sample data in `crossfade.station` (8 rows), read through these columns: `created_on, maximum_ambiguity, minimum_ambiguity, model_id`. Brief requirements met: (a) (b) (c) (e).

```
SELECT model_id,
       COUNT(*) AS stations,
       ROUND(AVG(maximum_ambiguity - minimum_ambiguity), 2) AS avg_window,
       SUM(CASE WHEN maximum_ambiguity - minimum_ambiguity > 0.5 THEN 1 ELSE 0 END) AS wide_stations,
       MIN(created_on) AS oldest_station
FROM crossfade.station
GROUP BY model_id
ORDER BY stations DESC,
         model_id;
```

EXPECTED OUTPUT: 4 row(s), in this order

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS An ambiguity window of exactly 0.50 is not counted as wide.

Why: 'wide' means more than half the range; the boundary itself is not in the sample data.

Input:

```
INSERT into station: station_id=7799, listener_id=20938, model_id=3, name=Edge Window, minimum_ambiguity=0.25, maximum_ambiguity=0.75, created_on=2026-09-01
```

Expected: model 3 (only station 7799, window 0.25 to 0.75): 1 station, avg_window 0.50, 0 wide.

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20
3	1	0.50	0	2026-09-01

UC-09 Find scoring results that predate a model redeployment

Needs track_score and scoring_model together (Task 3).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track_score`	`track_id, model_id, scored_on`	27 rows	with a join
`scoring_model`	`model_id, deployed_on`	5 rows	with a join

Input / expected output: not defined yet. This use case is answered in multiple_table.sql or routines.sql; its query, input and expected output are added when that file and its tests are written.

UC-10 Define stations by ambiguity window; measure how long they stay in use

Question: How adventurous is each station, how long has it been in use, and which model does it rely on?

The blend weights (rule I1) live in station_blend (Task 3).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`station`	`station_id, listener_id, model_id, name, minimum_ambiguity, maximum_ambiguity, created_on`	8 rows	alone: full
`station_blend`	`station_id, genre_id, weight`	19 rows	with a join

S6 | crossfade.station | single_table.sql

BUSINESS NEED

every station is evaluated against one nominated model version inside an ambiguity window (RA: Station; Scoring Model 1:N Station), so retiring or redeploying a version changes the stations that rely on it.

QUESTION

How many stations does each model version serve, how wide are their ambiguity windows, and since when has each version been relied on?

INPUT

The Project 1 sample data in `crossfade.station` (8 rows), read through these columns: `created_on, maximum_ambiguity, minimum_ambiguity, model_id`. Brief requirements met: (a) (b) (c) (e).

```
SELECT model_id,
       COUNT(*) AS stations,
       ROUND(AVG(maximum_ambiguity - minimum_ambiguity), 2) AS avg_window,
       SUM(CASE WHEN maximum_ambiguity - minimum_ambiguity > 0.5 THEN 1 ELSE 0 END) AS wide_stations,
       MIN(created_on) AS oldest_station
FROM crossfade.station
GROUP BY model_id
ORDER BY stations DESC,
         model_id;
```

EXPECTED OUTPUT: 4 row(s), in this order

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20

TEST CASES: data the sample never reaches; each runs in a transaction that is rolled back

PASS An ambiguity window of exactly 0.50 is not counted as wide.

Why: 'wide' means more than half the range; the boundary itself is not in the sample data.

Input:

```
INSERT into station: station_id=7799, listener_id=20938, model_id=3, name=Edge Window, minimum_ambiguity=0.25, maximum_ambiguity=0.75, created_on=2026-09-01
```

Expected: model 3 (only station 7799, window 0.25 to 0.75): 1 station, avg_window 0.50, 0 wide.

model_id	stations	avg_window	wide_stations	oldest_station
4	3	0.57	2	2026-06-10
2	2	0.60	2	2026-05-02
5	2	0.50	1	2026-07-20
1	1	0.60	1	2026-05-20
3	1	0.50	0	2026-09-01

UC-11 Place each recording between genres

Lives in the associative tables track_score and genre_probability (Task 3).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track_score`	`track_id, model_id, genre_id, top_probability, posterior_entropy, model_divergence, segment_variance`	27 rows	with a join
`genre_probability`	`track_id, model_id, genre_id, probability`	270 rows	with a join
`genre`	`genre_id, name`	10 rows	with a join

Input / expected output: not defined yet. This use case is answered in multiple_table.sql or routines.sql; its query, input and expected output are added when that file and its tests are written.

UC-12 Fill a station with recordings that match its blend and window

Five tables; the natural home is a procedure (Task 4).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`station`	`station_id, model_id, minimum_ambiguity, maximum_ambiguity`	8 rows	with a join
`station_blend`	`station_id, genre_id, weight`	19 rows	with a join
`track_score`	`track_id, model_id, posterior_entropy`	27 rows	with a join
`genre_probability`	`track_id, model_id, genre_id, probability`	270 rows	with a join
`track`	`track_id, title, duration, ingest_status`	15 rows	with a join

Input / expected output: not defined yet. This use case is answered in multiple_table.sql or routines.sql; its query, input and expected output are added when that file and its tests are written.

UC-13 Learn listener taste from completions, saves and skips

Lives in the associative table play (Task 3).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`play`	`listener_id, track_id, played_at, seconds_played, completed, reaction`	31 rows	with a join
`track`	`track_id, duration`	15 rows	with a join

Input / expected output: not defined yet. This use case is answered in multiple_table.sql or routines.sql; its query, input and expected output are added when that file and its tests are written.

UC-14 Keep an exact royalty ledger

play, track and artist together (Task 3 or 4).

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`play`	`listener_id, track_id, played_at, seconds_played, completed, royalty_amount`	31 rows	with a join
`track`	`track_id, artist_id, duration`	15 rows	with a join
`artist`	`artist_id, payout_rate`	8 rows	with a join

Input / expected output: not defined yet. This use case is answered in multiple_table.sql or routines.sql; its query, input and expected output are added when that file and its tests are written.

UC-15 Reject a duplicate upload

Question: What happens when an artist uploads an audio file that is already in the catalogue?

An error case: [E1], the last statement in single_table.sql, marked EXPECTED TO FAIL (MySQL must refuse it with 1062). Planned for routines.sql (Task 4): an upload procedure whose DECLARE ... HANDLER FOR 1062 catches the duplicate and answers the uploader instead of stopping.

DATA IT ACCESSES

Table	Columns	Sample data	Single table?
`track`	`track_id, artist_id, title, duration, audio_checksum`	15 rows	alone: full

E1 | crossfade.track | single_table.sql

BUSINESS NEED

a re-upload of the same audio file must never become a second catalogue entry earning a second royalty stream; the RA keeps the audio checksum (SHA-256 of the file) to "detect duplicate uploads" (RA: Track: Audio Checksum).

INPUT: a write the database must refuse (EXPECTED TO FAIL, the last statement in the file)

```
INSERT INTO crossfade.track (track_id, artist_id, title, duration, release_date, audio_checksum, ingest_status)
SELECT 9999, artist_id, CONCAT(title, ' (re-upload)'), duration, '2026-09-30', audio_checksum, 'awaiting scoring'
FROM crossfade.track
WHERE track_id = 4817;
```

EXPECTED OUTPUT

MySQL raises error **1062 uq_track_audio_checksum**; crossfade.track keeps its 15 rows. Test run: 1062 Duplicate entry '...' for key 'track.uq_track_audio_checksum'